

Lecture 16: Basic methods for IVPs

Jamie Haddock

Table of contents

1	Initial-value problems	1
1.1	Basics of IVPs	1
1.2	Numerical solutions	1
1.3	Existence	3
1.4	Uniqueness	4
1.5	Conditioning of first-order IVPs	4
1.6	Euler's Method	6
1.7	Algorithm: Euler's method for an IVP	6
1.8	Error	7
1.9	Local truncation error	7
1.10	Convergence	8

1 Initial-value problems

Definition: Initial-value problem (scalar)

A scalar first-order **initial-value problem** (IVP) is

$$\begin{aligned}u'(t) &= f(t, u(t)), & a \leq t \leq b \\ u(a) &= u_0.\end{aligned}$$

We call t the **independent variable** and u the **dependent variable**. If $u' = f(t, u) = g(t) + uh(t)$, the differential equation is **linear**; otherwise, it is **nonlinear**.

A **solution** of an initial-value problem is a function $u(t)$ that makes both $u'(t) = f(t, u(t))$ and $u(a) = u_0$ true equations.

1.1 Basics of IVPs

Linear IVPs can be solved in terms of integrals using an *integrating factor*. However, often the necessary integrals cannot be computed in closed form.

Nonlinear IVPs are typically even more challenging.

Most often, there is no analytic formula available for the solution of an IVP.

1.2 Numerical solutions

Numerical techniques for IVPs do not use techniques you may have learned in Differential Equations to produce analytic formulas for the solution. Instead, they approximate power series, integrals, and derivatives locally and use iterative progression to advance the independent variable and approximate a solution.

While numerical techniques are approximative, they are far more robust than analytic techniques in the scope of problems they can solve.

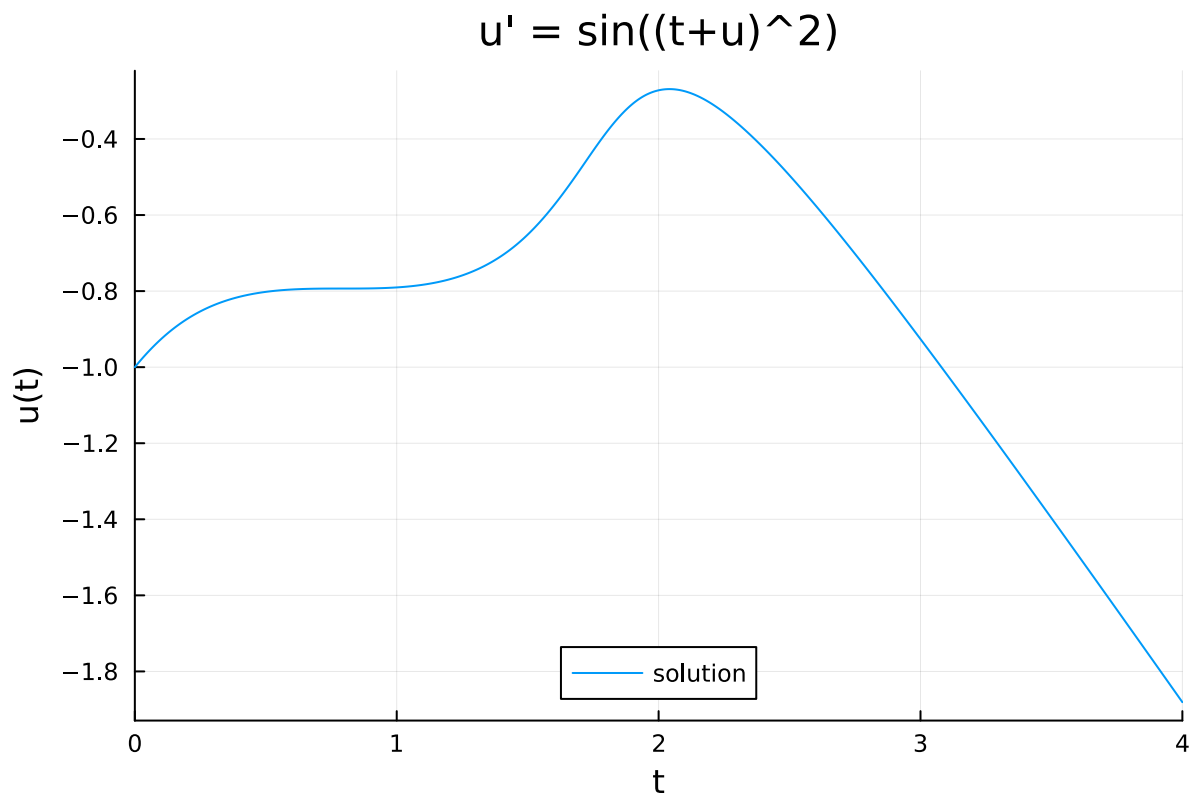
WARNING: using LinearAlgebra.rotate! in module Main conflicts with an existing identifier.

```
f = (u,p,t) -> sin((t+u)^2)    # defines du/dt, must include p -- not used here
u = -1.0                        # initial value
tspan = (0.0,4.0)              # t interval
```

```
(0.0, 4.0)
```

```
ivp = ODEProblem(f,u,tspan)
sol = solve(ivp,Tsit5());
```

```
plot(sol,label="solution",legend=:bottom,xlabel="t",ylabel="u(t)",title="u' = sin((t+u)^2)")
```



```
@show sol(1.0);    # sol acts like a function
[sol.t sol.u]      # under the hood, sol is defined by u values
                  # at t nodes and is interpolated
```

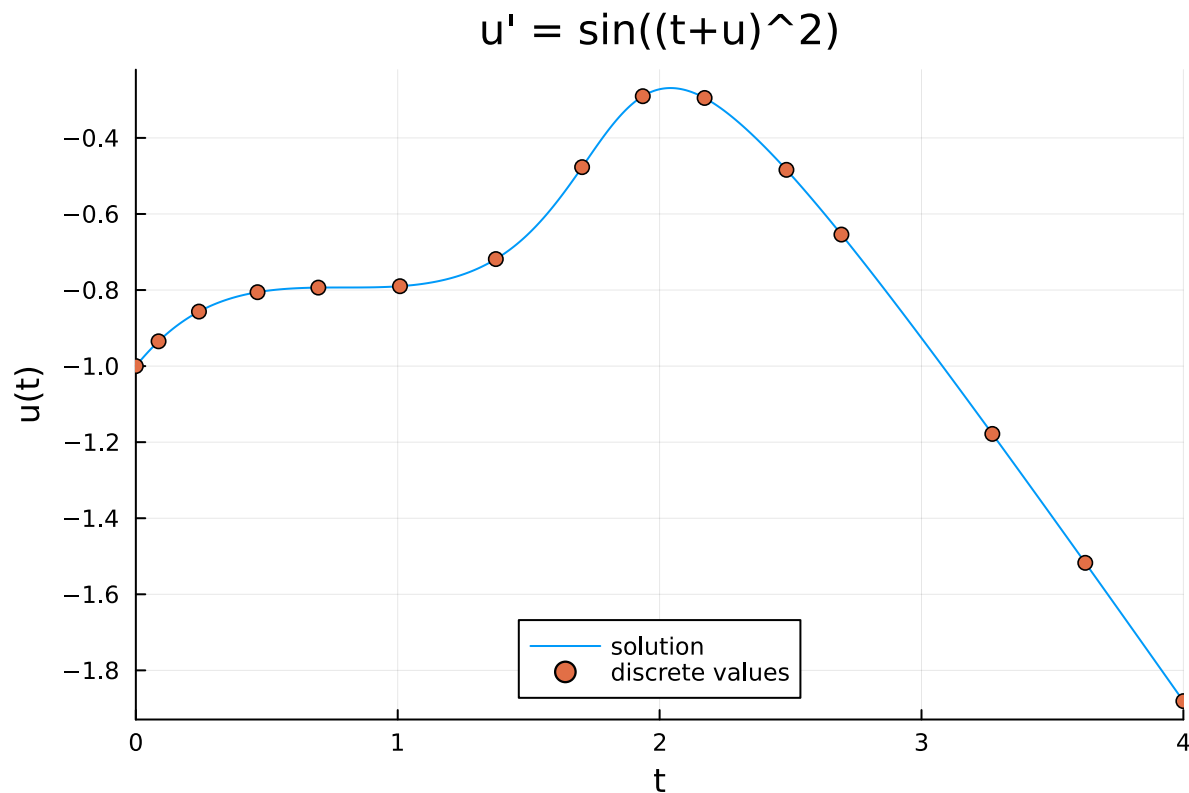
```
sol(1.0) = -0.7903205813665345
```

```
15×2 Matrix{Float64}:
```

```
0.0      -1.0
0.0867807 -0.93483
0.241035  -0.856617
0.464665  -0.805668
0.696832  -0.793614
```

1.00862	-0.789925
1.37461	-0.718601
1.70407	-0.476837
1.93572	-0.29033
2.17184	-0.294994
2.4843	-0.483948
2.69425	-0.654121
3.27049	-1.1783
3.62534	-1.51729
4.0	-1.88086

```
scatter!(sol.t,sol.u,label="discrete values")
```



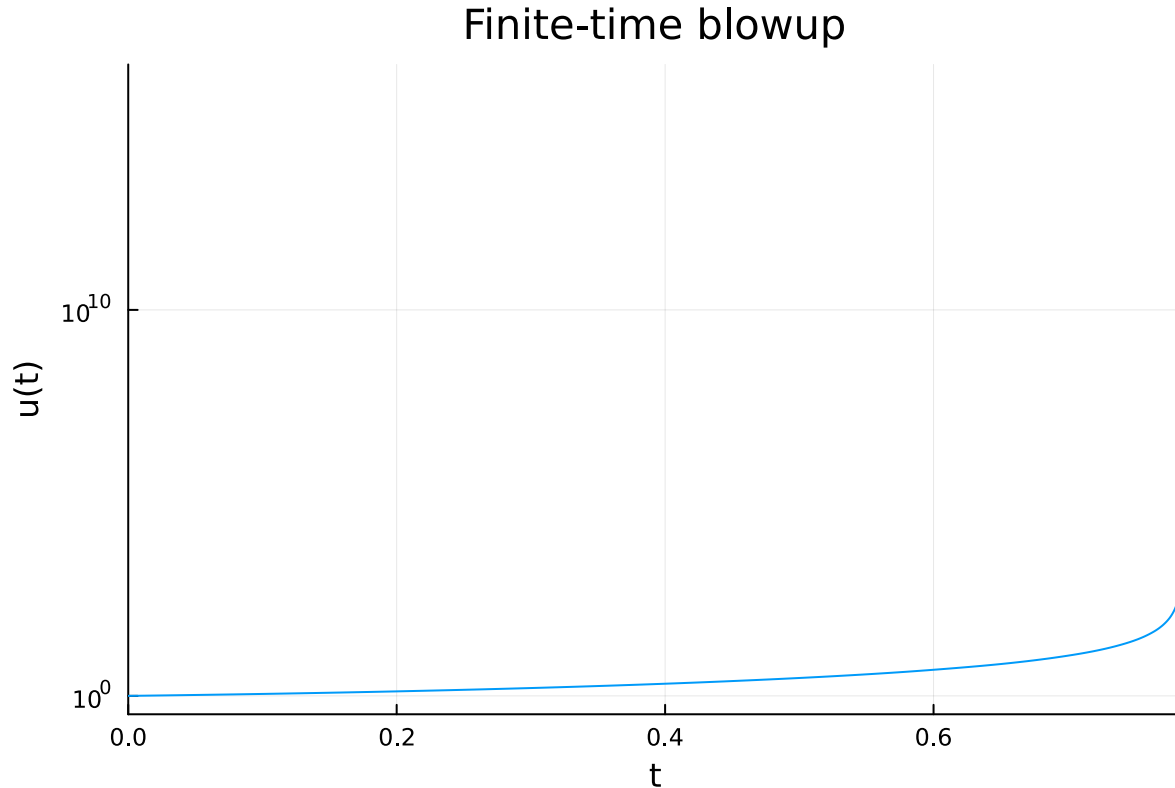
1.3 Existence

There are some simple IVPs that do not have solutions at all possible times.

```
f = (u,p,t) -> (t+u)^2
ivp = ODEProblem(f,1.0,(0.,1.))
sol = solve(ivp,Tsit5());
```

Warning: At t=0.7853839417697203, dt was forced below floating point epsilon 1.1102230246251565e-16, a
 @ SciMLBase ~/.julia/packages/SciMLBase/XzPx0/src/integrator_interface.jl:623

```
plot(sol,label="",xlabel="t",yaxis=(log10,"u(t)"),title="Finite-time blowup")
```



1.4 Uniqueness

IVPs may also have more than one solution.

However, there is a condition we can check that guarantees that a unique solution exists.

Theorem: Existence and uniqueness

If the derivative $\frac{\partial f}{\partial u}$ exists and $\left| \frac{\partial f}{\partial u} \right|$ is bounded by a constant L for all $a \leq t \leq b$ and all u , then the initial-value problem defined by f has a unique solution for $t \in [a, b]$.

1.5 Conditioning of first-order IVPs

We'll focus on the effect of perturbations to u_0 and not to $f(t, u)$.

Theorem: Dependence on initial value

If the derivative $\frac{\partial f}{\partial u}$ exists and $\left| \frac{\partial f}{\partial u} \right|$ is bounded by a constant L for all $a \leq t \leq b$ and all u , then the solution $u(t; u_0 + \delta)$ of $u' = f(t, u)$ with initial condition $u(0) = u_0 + \delta$ satisfies

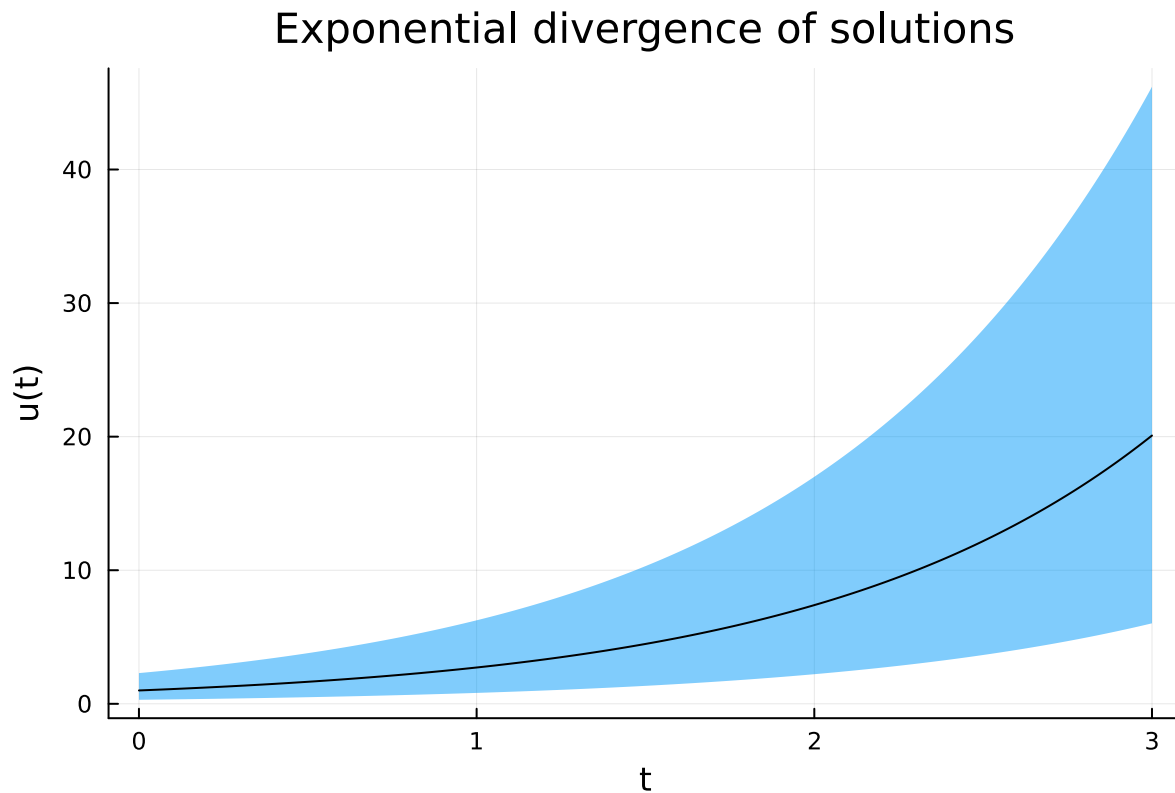
$$\|u(t; u_0 + \delta) - u(t; u_0)\|_{\infty} \leq |\delta| e^{L(b-a)}$$

for all sufficiently small $|\delta|$.

This theorem gives $e^{L(b-a)}$ as an upper bound on the infinity norm absolute condition number of the solution with respect to perturbations to the IVP initial value.

However, like we've seen before, these bounds can be *very* rough.

```
t = range(0,3,length=800)
u = @. exp(t)*1'
lower, upper = @. exp(t)*0.7, @. exp(t)*1.3
plot(t,u,l=:black,ribbon=(lower,upper), leg=:none,xlabel="t",ylabel="u(t)",
      title="Exponential divergence of solutions")
```



This IVP has condition number equal to the upper bound provided in the theorem above.

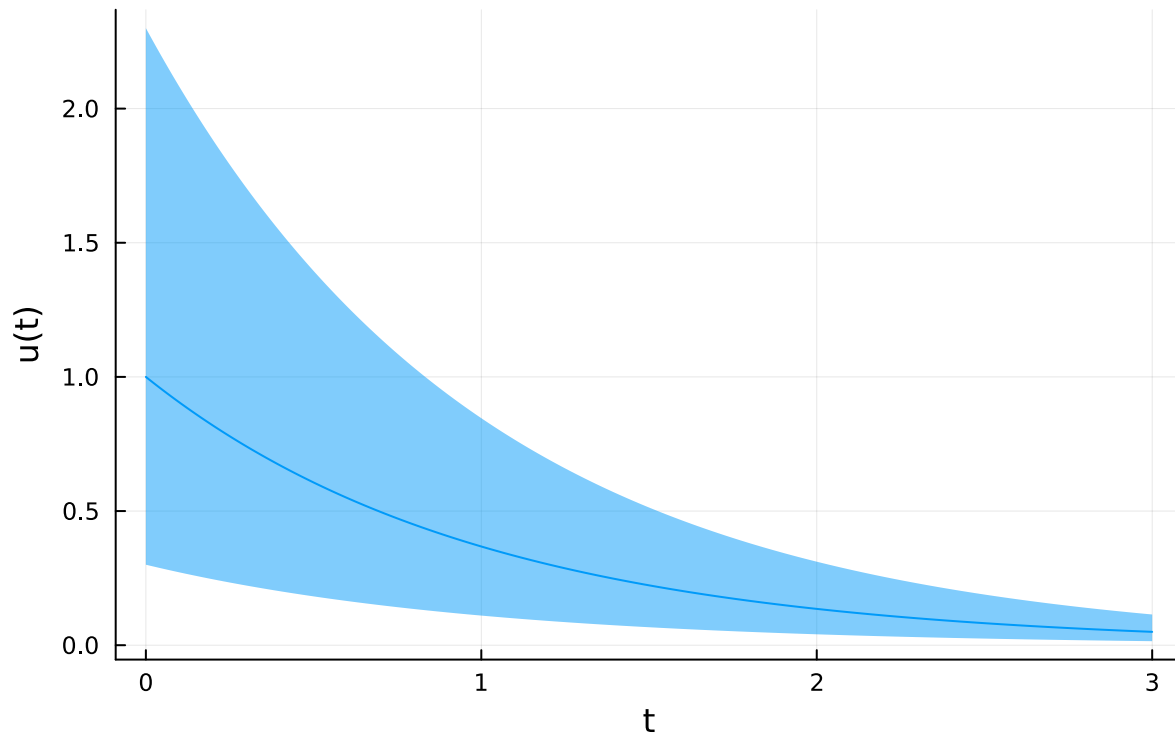
But with $u' = -u$, solutions actually get closer over time, so the maximum deviation of the solution is the same as the perturbation to the initial value.

```
u = @. exp(-t)*1
lower, upper = @. exp(-t)*0.7, @. exp(-t)*1.3
plot(t,u,l=:black,ribbon=(lower,upper),leg=:none,xlabel="t",ylabel="u(t)",
      title="Exponential convergence of solutions")
```

Warning: Skipped line arg bloack.

@ Plots ~/.julia/packages/Plots/FFuQi/src/args.jl:1105

Exponential convergence of solutions



In this case, the condition number is one. Here, the exponentially growing bound is a terrible overestimate!

1.6 Euler's Method

One way to estimate a solution to an IVP is to estimate its values at a finite collection of nodes:

$$t_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

The number h is called the **step size**.

We'll let $\hat{u}(t)$ denote the exact solution of the IVP and our approximations at t_i be $u_i \approx \hat{u}(t_i)$.

The model that defines Euler's method is asking that the piecewise linear interpolant of the values $u_0, \dots, u_n$ have slopes equal to the derivatives of the exact solution at the point (t_i, u_i) ,

$$\frac{u_{i+1} - u_i}{h} = \frac{u_{i+1} - u_i}{t_{i+1} - t_i} = f(t_i, u_i).$$

1.7 Algorithm: Euler's method for an IVP

Given the IVP $u' = f(t, u)$, $u(a) = u_0$ and the nodes $t_i = a + ih$, $h = (b-a)/n$, iteratively compute the sequence

$$u_{i+1} = u_i + hf(t_i, u_i), \quad i = 0, \dots, n-1.$$

```
"""
    euler(ivp,n)

Apply Euler's method to solve the given IVP using 'n' time steps.
Returns a vector of times and a vector of solution values.
"""
```

```

function euler(ivp,n)
    # Time discretization
    a,b = ivp.tspan
    h = (b-a)/n
    t = [ a + i*h for i in 0:n ]

    # Initial condition and output setup.
    u = fill(float(ivp.u0),n+1)

    # The time stepping criterion
    for i in 1:n
        u[i+1] = u[i] + h*ivp.f(u[i],ivp.p,t[i])
    end
    return t,u
end

```

euler

This is an example of a one-step method.

Definition: One-step IVP method

A **one-step method** is a formula of the form

$$u_{i+1} = u_i + h\phi(t_i, u_i, h), \quad i = 0, \dots, n-1.$$

1.8 Error

Suppose we have access to the exact value at t_i , $\hat{u}(t_i) = u_i$. How good is our u_{i+1} approximation to $\hat{u}(t_{i+1})$?

Exercise:

Using Taylor series expansion, simplify $\hat{u}(t_{i+1}) - [u_i + hf(t_i, u_i)]$.

Answer:

We have

$$\begin{aligned}
 \hat{u}(t_{i+1}) - [u_i + hf(t_i, u_i)] &= \hat{u}(t_{i+1}) - [\hat{u}(t_i) + hf(t_i, \hat{u}(t_i))] \\
 &= [\hat{u}(t_i) + h\hat{u}'(t_i) + \frac{h^2}{2}\hat{u}''(t_i) + O(h^3)] - [\hat{u}(t_i) + h\hat{u}'(t_i)] \\
 &= \frac{h^2}{2}\hat{u}''(t_i) + O(h^3)
 \end{aligned}$$

1.9 Local truncation error

Definition: Truncation error of a one-step IVP method

The **local truncation error** (LTE) of a one-step method is

$$\tau_{i+1}(h) = \frac{\hat{u}(t_{i+1}) - \hat{u}(t_i)}{h} - \phi(t_i, \hat{u}(t_i), h).$$

The method is called **consistent** if $\tau_{i+1}(h) \rightarrow 0$ as $h \rightarrow 0$.

Note that the local error in stepping from t_i to t_{i+1} is $h\tau_{i+1}(h)$. We can make these local errors smaller by chopping h in half, but we would have to take twice as many steps. The important thing to measure is local error *per unit step length*, which is how τ is defined.

Lemma:

If $\phi(t, u, 0) = f(t, u)$ for any function u , then the method defined by ϕ is consistent.

1.10 Convergence

Definition: Global error of an IVP solution

Given an IVP whose exact solution is $\hat{u}(t)$, the **global error** of approximate solution values $u_0, u_1, \dots, u_n$ at times t_i is the vector $[\hat{u}(t_i) - u_i]_{i=0, \dots, n}$.

One might hope that the global error simply adds up all of the local errors over the steps taken, however each step causes a perturbation of the solution which then grows.

Theorem:

Suppose that the unit local truncation error of the one step method satisfies

$$|\tau_{i+1}(h)| \leq Ch^p,$$

and that

$$\left| \frac{\partial \phi}{\partial u} \right| \leq L$$

for all $t \in [a, b]$, all u , and all $h > 0$. Then the global error satisfies

$$|\hat{u}(t_i) - u_i| \leq \frac{Ch^p}{L} [e^{L(t_i-a)} - 1],$$

which is $O(h^p)$ as $h \rightarrow 0$.

Definition: Order of accuracy of a one-step IVP method

If the local truncation error of the one-step method satisfies $\tau_{i+1}(h) = O(h^p)$ for a positive integer p , then p is the **order of accuracy** of the formula.

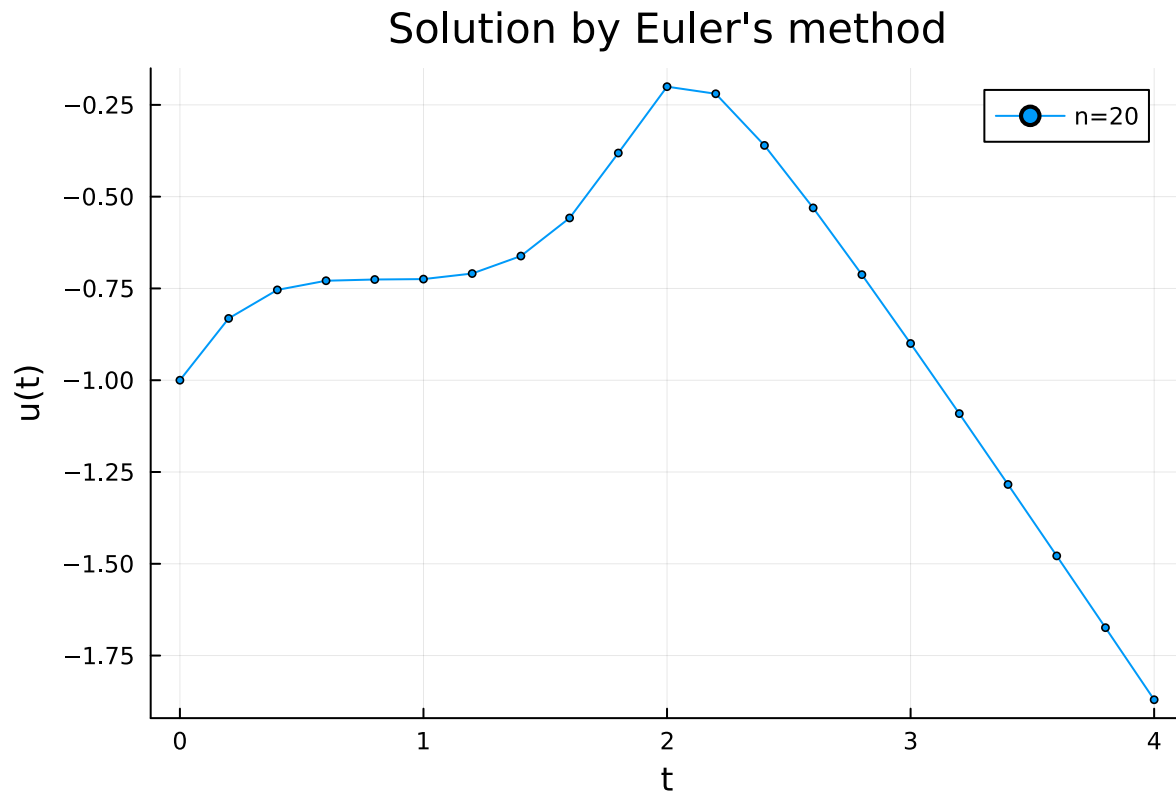
```
f = (u,p,t) -> sin((t+u)^2);  
tspan = (0.0,4.0);  
u0 = -1.0;
```

```
ivp = ODEProblem(f,u0,tspan)
```

```
ODEProblem with uType Float64 and tType Float64. In-place: false  
timespan: (0.0, 4.0)  
u0: -1.0
```

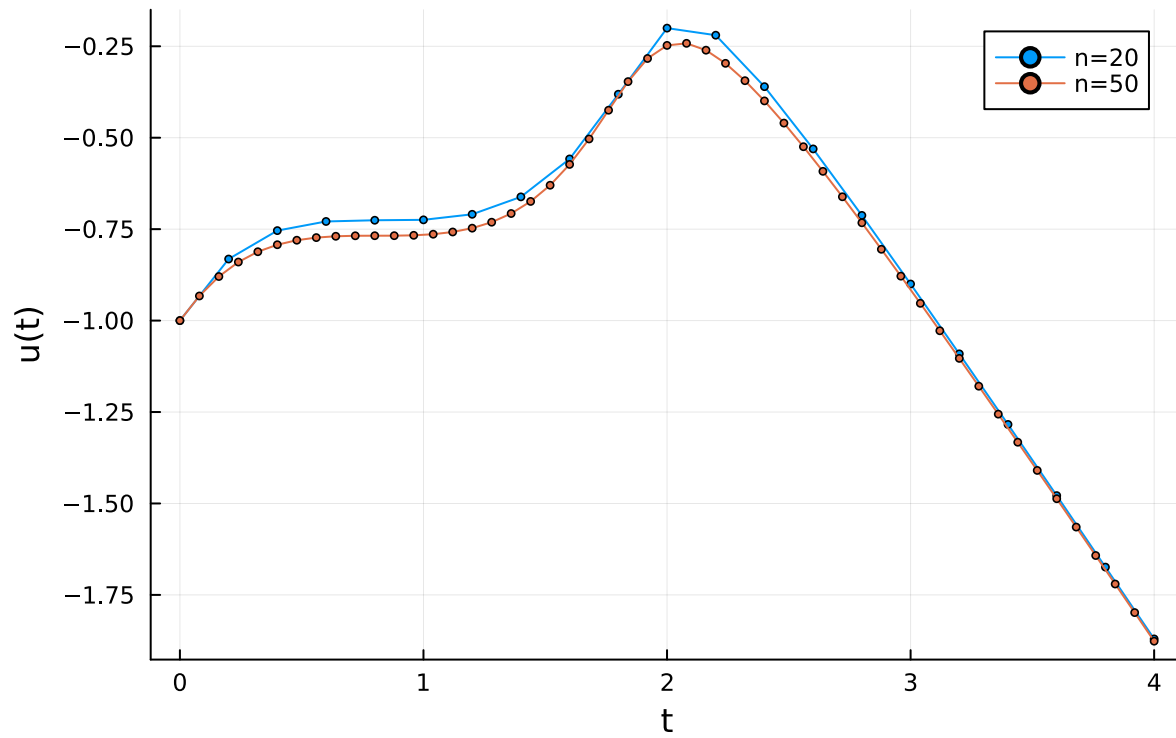
```
t,u = euler(ivp,20)
```

```
plot(t,u,m=2,label="n=20",xlabel="t",ylabel="u(t)",  
      title="Solution by Euler's method")
```



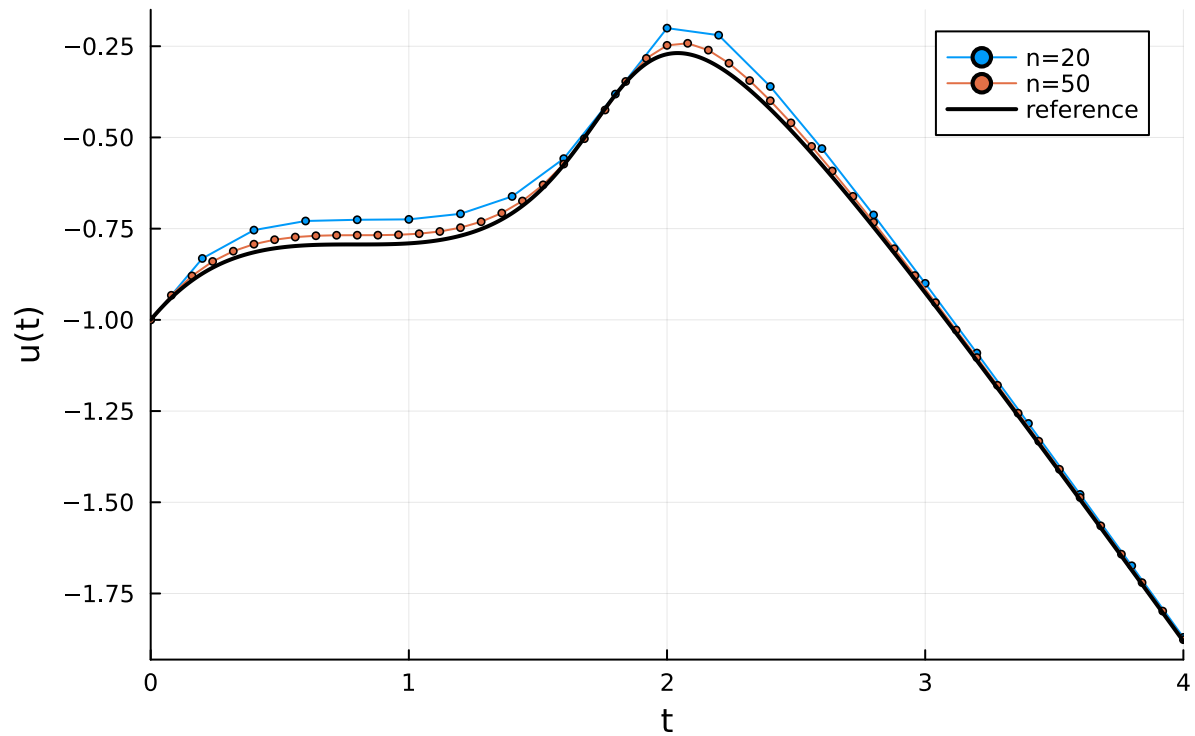
```
t,u = euler(ivp,50)  
plot!(t,u,m=2,label="n=50")
```

Solution by Euler's method



```
u_exact = solve(ivp,Tsit5(),reltol=1e-14,abstol=1e-14)
plot!(u_exact,l=(2,:black),label="reference")
```

Solution by Euler's method



```
n = [ round(Int,5*10^k) for k in 0:0.5:3 ]
errs = []
for n in n
    t,u = euler(ivp,n)
    push!(errs, norm(u_exact.(t)-u, Inf) )
end

plot(n,errs,m=:o,label="results",xaxis=(log10,"n"),
     yaxis=(log10,"Inf-norm global error"),
     title="Convergence of Euler's method")
plot!(n,0.05*(n/n[1]).^(-1),l=:dash,label="O(n^{-1})")
```

Convergence of Euler's method

